NAME :G.AKSHAYA

REG NO: 192521156

COURSE CODE : CSA0619

COURSE NAME : DESIGN AND ANALYSIS OF ALGORITHMS

CO4 - ASSESSMENT TOOL – 03

Q2. Shortest Path Comparison (Dijkstra vs Bellman-Ford) :

Aim:

To compare Dijkstra's algorithm and Bellman-Ford algorithm for finding shortest paths in a weighted graph containing negative edge weights, and analyze their correctness, complexity, performance, strengths, and limitations.

Problem Statement:

Given a weighted graph containing negative edge weights, apply both Dijkstra's and Bellman-Ford algorithms to compute shortest paths. Compare their results and analyze why Dijkstra fails in certain cases, while Bellman-Ford can correctly handle negative edge weights.

Objective:

- To find the shortest paths from a source vertex to all other vertices.
- To implement Dijkstra’s algorithm and Bellman-Ford algorithm.
- To compare their correctness and performance.
- To analyze how both algorithms handle negative edge weights.
- To understand why Dijkstra’s algorithm fails with negative weights.
- To compare their time complexity and limitations.
- To identify the appropriate algorithm based on the type of graph and edge weights.

Graph Used :

A --2--> B

A --5--> C

C --(-4)--> B

Starting vertex: **A**

Possible Paths to B

A → B = 2

A → C → B

= 5 + (-4)

= 1

Therefore, the actual shortest distance from **A to B is 1**.

Algorithm 1: Dijkstra:

1. Set the source distance to 0.
2. Set all other distances to infinity.
3. Select the unvisited vertex with the smallest distance.
4. Relax its adjacent edges.
5. Mark the selected vertex as visited.
6. Repeat until all required vertices are processed.
7. Display shortest distances.

Limitation:

Dijkstra's algorithm assumes non-negative edge weights. With negative edges, a vertex may be finalized too early.

Algorithm 2: Bellman-Ford

1. Set source distance to 0.
2. Set all other distances to infinity.
3. Relax every edge `V-1` times.
4. Update the shortest distance whenever a shorter path is found.
5. Perform one additional relaxation to detect a negative cycle.

6. Display shortest distances.

Bellman-Ford :

It can handle negative edge weights and can also detect negative

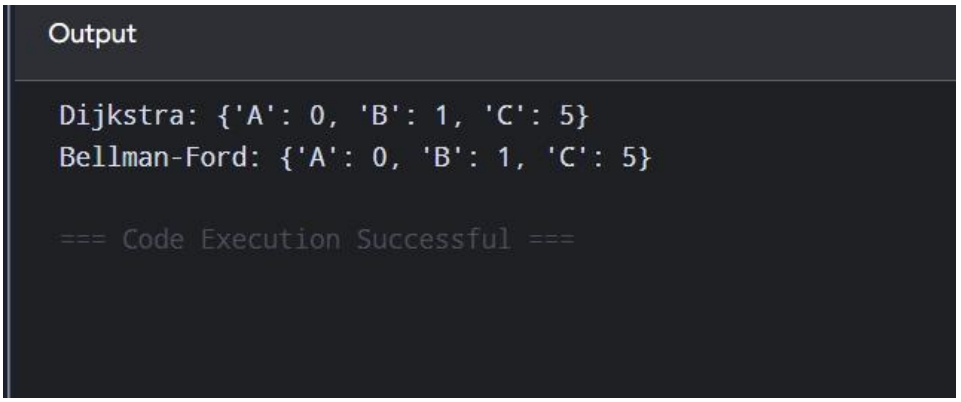
SOURCE CODE:

main.py

```
INF=float('inf')
edges=[('A','B',2),('A','C',5),('C','B',-4)]
vertices=['A','B','C']
def dijkstra():
    dist={v:INF for v in vertices}
    dist['A']=0
    used=set()
    for _ in vertices:
        u=min((v for v in vertices if v not in used
        used.add(u)
        for a,b,w in edges:
            if a==u and dist[a]+w<dist[b]:
                dist[b]=dist[a]+w
    return dist
def bellman_ford():
    dist={v:INF for v in vertices}
```

```
    return dist
def bellman_ford():
    dist={v:INF for v in vertices}
    dist['A']=0
    for _ in range(len(vertices)-1):
        for a,b,w in edges:
            if dist[a]!=INF and dist[a]+w<dist[b]:
                dist[b]=dist[a]+w
    return dist
print("Dijkstra:",dijkstra())
print("Bellman-Ford:",bellman_ford())
```

OUTPUT:



Correct Shortest Path :

A → C → B

Cost = 5 + (-4)

= 1

Hence, Bellman-Ford gives the correct shortest distance of 1, while Dijkstra gives 2 for this negative-edge example.

Comparative Analysis :

Feature	Dijkstra	Bellman-Ford
Negative edges	Not suitable	Suitable
Negative cycle detection	No	Yes
Basic approach	Greedy	Dynamic Programming/Relaxation
Time Complexity	$O(V^2)$	O(VE)
Speed	Faster	Slower
Accuracy with negative edges	Can fail	Correct
Suitable	Non-negative weights	Graphs with negative weights

Why Dijkstra Fails

Dijkstra permanently selects the currently smallest distance.

In this example:

A → B = 2

A → C = 5

Dijkstra may finalize **B = 2** before discovering:

A → C → B = 5 + (-4) = 1

Therefore, Dijkstra cannot guarantee the correct result when negative edges are present.

Advantages :

Dijkstra

- Fast for non-negative weighted graphs.
- Simple to implement.
- Efficient for large graphs.
- Suitable for routing and network applications.

Bellman-Ford

- Handles negative edge weights.
- Detects negative-weight cycles.
- Provides correct shortest paths in graphs with negative edges.

Limitations:

Dijkstra

- Cannot correctly handle general negative edge weights.
- Cannot detect negative-weight cycles.

- Requires non-negative edge weights.

Bellman-Ford

- Slower than Dijkstra.
- Higher time complexity.
- Less suitable for very large dense graphs.

Result:

The comparison shows that Dijkstra is faster but requires non-negative edge weights, whereas Bellman-Ford is more flexible because it can handle negative edge weights and detect negative cycles. Therefore, Dijkstra should be selected for graphs with non-negative weights, while Bellman-Ford should be used when negative edge weights are present. This directly addresses the comparative-analysis requirements in the attached question.